Site 6457: A Prisoner's Dilemma – Team Proposal

Truman Hess Josh Smith Larry Wilson Wafer Sung Michael Markoe

September 22, 2025

1 Basic Gameplay & Core Mechanics

Core Gameplay Loop. The player toggles between four prisoners to pathfind and escape a procedurally generated horror environment. Each prisoner must evade the monster, locate the exit, and *stay alive*. The player alternates control to coordinate escapes while non-selected characters are controlled by simple AI. The monster evolves within a round by absorbing traits from captured prisoners, and the next round's difficulty is influenced by how many (and which types of) prisoners were saved previously.

Unique Gameplay Elements.

- *Monster Evolution*: When the monster captures a prisoner, it gains the strengths and weaknesses of that prisoner. Potential evolutions include speed, pathfinding effectiveness, and environmental manipulation (e.g., doors and entry points).
- *Procedural Levels*: Layouts vary each run.
- *Unlockable Control*: Potential variant—start as a single prisoner; others become controllable only after they are found.

2 Formal Elements of the Game

a) Players & Interaction Modes

- **Number of players**: MVP single-player; future potential for multiplayer.
- **Interaction style**: MVP toggling between prisoners; future co-op possibilities.

b) Objectives

- **Primary**: Escape with at least one prisoner.
- **Secondary / stretch**: Escape with multiple prisoners.
- **Perfect win**: Escape with all prisoners.

c) Procedures

- **Starting action**: Player spawns alone in a room with a note.
- **Progression**: Explore the generated environment and find other prisoners who can aid in escape. Audio cues indicate monster proximity.
- **Special actions**:
  - Each prisoner has a unique skill: (1) speed, (2) environmental manipulation (doors), (3) enhanced hearing.
  - If the monster catches a prisoner off-screen, the player is notified via sound effects.
  - On capture, the monster "trains" for 5–15 seconds, absorbing that prisoner's skill (e.g., becomes faster after capturing the speed-focused prisoner).
- **Resolving action**: One to four prisoners reach the exit before all are captured.

d) Rules

- **Objects & concepts**: Characters, monster, environment; the player can move, jump, toggle between characters, and interact with objects (e.g., doors).

- **Restrictions**: The player can command characters to wait or to follow the actively controlled character.
- **Effects**: Non-controlled prisoners generally stay put; depending on attributes, they may hide effectively from the monster.

e) Resources

- **From the monster's perspective**: Prisoners are resources that power the monster up (or down) by absorbing their skills.
- **Exit points**: Initially one exit (may scale with difficulty later).
- **From the player's perspective**: Prisoners are resources—adding prisoners improves team capabilities and score.

f) Conflict

- **Opponent**: The monster.
- **Obstacles**: Doors and room access points.
- **Player dilemma**: If the exit is found before all prisoners are located, decide whether to escape now or keep searching; optionally "sacrifice" a prisoner by ordering them to stay hidden.

g) Boundaries

- **Map**: Procedurally generated horror setting (e.g., prison or hospital).
- **Access points**: Some walls are breakable only by certain prisoner types; some doors can be unlocked only by others.

h) Outcome

- **Win**: Highest win is all four escape.
- **Partial win**: One to three escape.
- **Loss**: All are captured by the monster.

3 Narrative / Story Components

- **Theme / Setting**: Horror atmosphere (hospital or prison).
- **Backstory**: Prisoners are test subjects in an AI training experiment; the monster learns by capturing escapees.
- **Atmosphere Goals**: Excitement and fear via sound and environment; constant trade-offs between personal survival and group success.

4 Work Plan

Team divides the game into key functional areas and generates tickets on a Trello board.

Functional Breakout

- AI Systems
- Procedural Generation
- Environmental
- Gameplay Control
- Audio and UI

Key Algorithms / Libraries

- Kruskal's algorithm (minimum spanning tree) to connect rooms.
- A* (A-star) for pathfinding.
- Behavior Trees for agent control.